Regime-Aware Machine Learning Trading System

Project Strategy, Research Review & Technical Roadmap

Dataset: SPY — 2010-01-04 to 2025-12-30 (4,023 trading days)

March 2026

Table of Contents

1. Executive Summary & Project Purpose

2. Research Question & Hypothesis

3. Literature Review

3.1 Market Regime Detection

3.2 Technical Pattern Detection

3.3 Machine Learning for Trading

3.4 Backtesting & Validation

3.5 Risk Management

3.6 Feature Engineering (de Prado)

4. System Architecture

5. Use Cases

6. Current State & Results

7. Roadmap & Implementation Plan

8. Recommendations & Suggestions

9. References

1. Executive Summary & Project Purpose

This project develops a **regime-aware machine learning trading system** that combines three pillars: (1) algorithmic technical pattern detection, (2) Hidden Markov Model-based market regime classification, and (3) supervised ML models that learn regime-conditional trading signals. The core insight is that financial markets are **non-stationary** — a strategy calibrated to bull-market dynamics will systematically fail in bear markets. By explicitly modelling regime switches, the system adapts its signal generation, position sizing, and risk limits to the current market environment.

The system is being developed in phases. **Phase 0** (data pipeline) and **Phase 1** (pattern detection with ATR-based adaptive thresholds) are complete. The pattern detectors have been tuned to a combined event rate of **27.3%**, within the 10–30% target band. The next phases involve feature engineering, HMM regime detection, ML signal modelling, and walk-forward backtesting.

Purpose

- Build a systematic, evidence-based trading framework that outperforms regime-agnostic baselines.
- Investigate whether HMM-based regime detection provides statistically significant alpha when combined with technical pattern signals.
- Create a reusable, modular Python codebase suitable for academic research and practical deployment.
- Document all methodology with thesis-grade rigour for reproducibility.

2. Research Question & Hypothesis

Research Question: Does combining HMM-based market regime detection with event-driven technical signal selection improve out-of-sample risk-adjusted trading performance on U.S. equity markets?

Hypothesis H1: A regime-aware signal model (XGBoost conditioned on HMM regime labels + pattern features) achieves a higher out-of-sample Sharpe ratio than the same model without regime labels.

Hypothesis H2: Regime-conditional position sizing (reducing exposure in bear regimes) reduces maximum drawdown by at least 20% compared to fixed sizing, without proportionally reducing returns.

Null Hypothesis H0: Regime labels carry no incremental predictive information beyond what is already captured by volatility and momentum features.

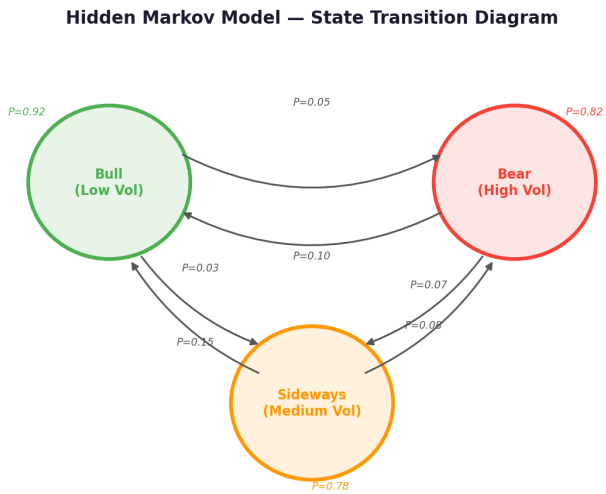
Evaluation Framework

Metric	Target	Benchmark
Sharpe Ratio (OOS)	> 1.0	Buy & Hold SPY (~0.7)
Max Drawdown	< 20%	SPY MDD (~34%)
Hit Rate	> 52%	Random (50%)
Calmar Ratio	> 0.5	SPY (~0.3)
H1 Ablation	Regime model > No-regime	Same features, no HMM

3. Literature Review

3.1 Market Regime Detection

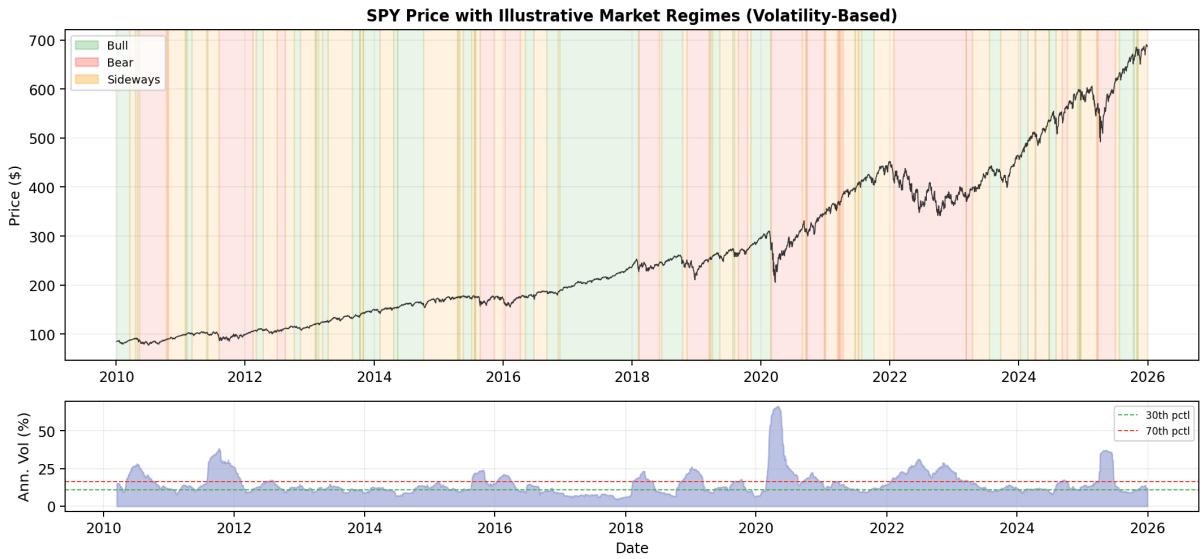
Market regimes are persistent, qualitatively distinct states of market behaviour characterised by differing return distributions, volatility levels, and correlation structures. The foundational work is **Hamilton (1989)**, which introduced Markov-switching autoregressive models for business cycle analysis. In finance, the canonical setup uses a Gaussian Hidden Markov Model (HMM) with 2–3 hidden states trained on daily log returns and realised volatility.



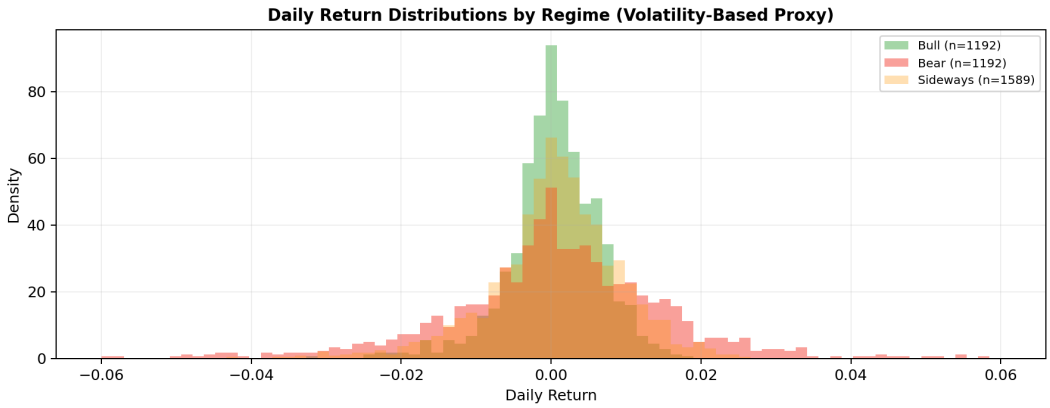
The diagram above illustrates a 3-state HMM with typical transition probabilities. Bull regimes are highly persistent (P=0.92), bear regimes less so (P=0.82), and sideways regimes serve as transitional states. The Viterbi algorithm decodes the most likely state sequence given observed returns.

Alternative Approaches

- **Statistical Jump Models** (Nystrup, Kolm & Lindstrom, 2020): temporal clustering with a jump penalty discouraging frequent regime switches. More robust than HMMs against poor EM initialisation and Gaussian emission misspecification.
- **VIX Threshold Models:** Simple rule-based classification (VIX<15 = bull, 15–25 = normal, >25 = bear). Transparent and lag-free but cannot capture latent structure.
- **Hybrid Ensemble-HMM** (Gupta et al., 2025): combines HMM with Random Forest and Gradient Boosting via a voting classifier. Achieves state-of-the-art regime detection accuracy by leveraging both generative and discriminative models.



The chart above shows SPY price with a volatility-based regime proxy. In the final system, an HMM will replace this heuristic with probabilistic regime labels.



Return distributions differ substantially across regimes. Bear regimes exhibit wider tails and negative skew, validating the need for regime-conditional risk management.

3.2 Technical Pattern Detection in Quantitative Finance

The algorithmic detection of chart patterns was formalised by **Lo, Mamaysky & Wang (2000)**, who used nonparametric kernel regression to identify head-and-shoulders, triangles, and channels across all NYSE/AMEX/Nasdaq stocks (1962–1996). They found that several patterns provide incremental information beyond random-walk null models. Earlier, **Brock, Lakonishok & LeBaron (1992)** showed that simple moving-average and trading-range-breakout rules produced returns inconsistent with standard null models on the DJIA (1897–1986).

Our Implementation

- **Support/Resistance:** Rolling max(High)/min(Low) with ATR-based proximity bands (0.3×ATR). Adapts to volatility; 50-bar window captures meaningful levels.
- **Triangles:** Linear regression on highs/lows to detect converging trendlines. Requires 3% compression and fires only on the breakout bar (0.3×ATR beyond recent range).
- **Channels:** Parallel trendlines validated with 5 filters: slope magnitude, parallelism (<15%), same direction, width (1–6×ATR), and minimum 3 touches per band.
- **Multiple Tops/Bottoms:** Rolling extremes with 3-bar close-trend confirmation via polyfit.

Criticisms & Mitigations

Data-snooping bias: With many patterns and parameters, some will appear profitable by chance. Mitigation: walk-forward validation and the Deflated Sharpe Ratio (Bailey et al., 2017). **Subjectivity:** Even algorithmic implementations have parameter degrees of freedom. Mitigation: use pattern signals as ML features rather than standalone rules, letting the model learn their value. **Diminishing edge:** Widely known patterns may lose profitability over time (Adaptive Markets Hypothesis, Lo 2004). Mitigation: regime conditioning adds a layer of differentiation.

3.3 Machine Learning Models for Trading

The model taxonomy for financial prediction spans traditional ML and deep learning. For tabular financial features (our case), gradient-boosted trees consistently outperform neural networks. The recommended architecture:

Model	Strengths	Best For
XGBoost / LightGBM	State-of-art tabular perf. Handles missing data, fast	Signal classification Return prediction
Random Forest	Robust to overfitting Built-in feature importance	Baseline model Feature selection
LSTM	Captures sequential dependencies Handles variable sequences	Volatility forecasting Medium-horizon returns
Stacked Ensemble	Combines best of each XGBoost as meta-learner	Production system Maximum accuracy

For our system, we recommend XGBoost as the primary signal model with a Random Forest baseline for ablation studies. Pattern signals, regime labels, and momentum/volatility features serve as inputs. Labels are generated via the Triple Barrier Method.

3.4 Backtesting & Validation

Walk-Forward Validation (WFO) is the gold standard for temporal data. The dataset is divided into sequential rolling windows: a training window (e.g., 3 years) followed by a test window (e.g., 6 months). The process ‘walks forward’ through time, retraining on each new window. This prevents look-ahead bias.

Combinatorial Purged Cross-Validation (CPCV), introduced by de Prado (2018), produces a distribution of performance metrics rather than a single backtest. Purging removes training observations that overlap temporally with test labels. Embargo adds a buffer period. This enables rigorous overfitting assessment.

Bias	Description	Mitigation
Look-ahead	Using future data in features	Strict temporal ordering
Survivorship	Testing only on surviving assets	Point-in-time databases
Data-snooping	Reporting best of many tests	Deflated Sharpe Ratio
Overfitting	Over-optimising to historical data	WFO + CPCV + regularisation
Unrealistic fills	Ignoring slippage/impact	Commission + slippage models

3.5 Risk Management in Regime-Aware Systems

The key innovation is **dynamic sizing conditional on the detected regime**. The Kelly Criterion defines optimal bet size as f* = edge/odds, but in practice fractional Kelly (half or less) is standard because edge estimates are uncertain. Thorp (2006) showed that betting double Kelly eliminates 100% of the gain.

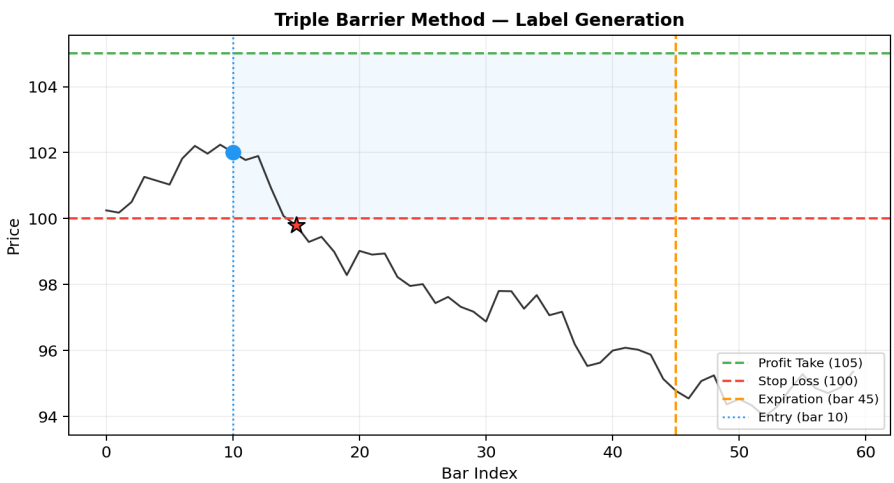
Regime	Strategy Emphasis	Position Sizing	Stops
Bull / Low Vol	Trend-following, momentum	Full target vol	Wider (1.5x ATR)
Bear / High Vol	Defensive, hedged	Half target vol	Tight (0.75x ATR)
Sideways	Mean-reversion, range-bound	Reduced size	Medium (1x ATR)

A **drawdown brake** mechanism progressively reduces position sizes at predefined drawdown thresholds (-5%, -10%, -15%) and halts all new positions at -20%. Full sizing is restored only after a confirmed 50% equity recovery.

3.6 Feature Engineering (de Prado)

Advances in Financial Machine Learning (de Prado, 2018) introduces several paradigm-shifting concepts for financial feature engineering:

- **Fractional Differentiation:** Apply a non-integer differencing operator d (typically ~0.2) to achieve stationarity while preserving >90% memory of the original series. Standard returns (d=1) destroy all memory.
- **Triple Barrier Method:** Labels each trade entry with +1 (profit-take hit), -1 (stop-loss hit), or 0 (expiration). Encodes risk-reward dynamics better than simple return signs.
- **Meta-Labeling:** Separates side prediction (long/short) from sizing (confidence). A secondary model predicts bet size given the primary model's signal.
- **CUSUM Filter:** Event-driven sampling that triggers only when cumulative returns deviate from zero by a threshold, improving signal-to-noise ratio.
- **Feature Importance:** MDI, MDA, and SFI methods for assessing feature relevance with proper handling of temporal dependencies.

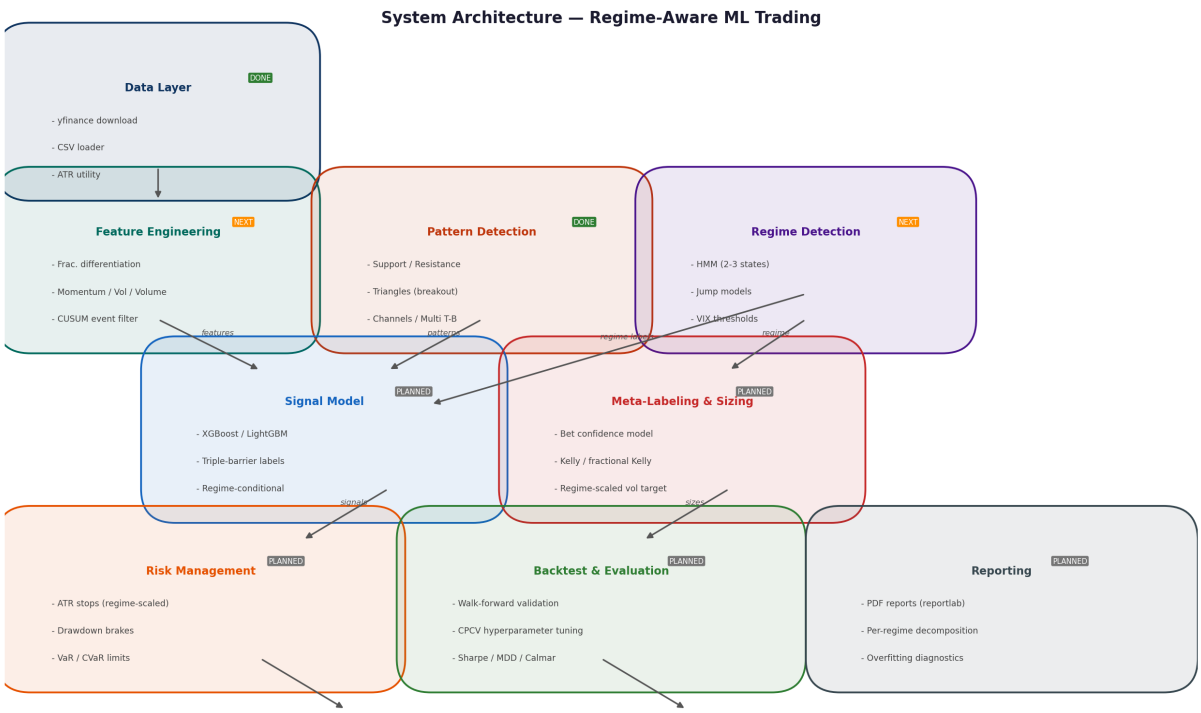


The Triple Barrier illustration above shows an entry at bar 10 with three boundaries: profit-take (green), stop-loss (red), and expiration (orange). The first barrier touched determines the label.

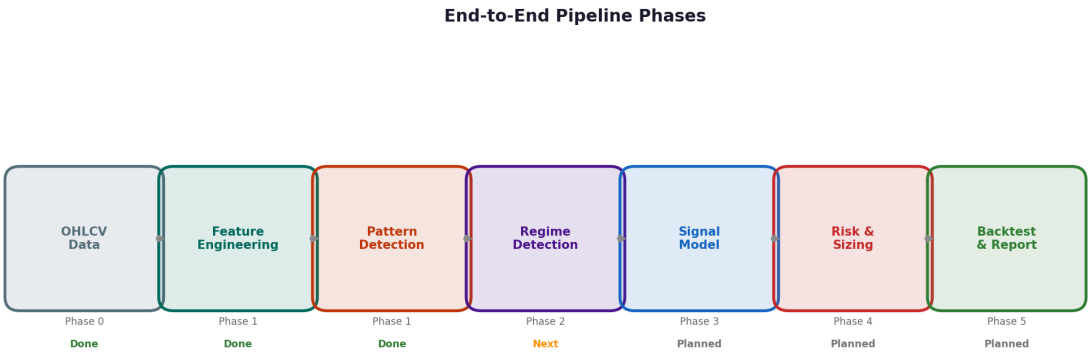
Category	Specific Features	Source
Returns	Frac. diff. prices (d~0.2), log returns 1d/5d/21d/63d	de Prado Ch. 5
Volatility	ATR, realised vol, Garman-Klass, Yang-Zhang	Standard + utils.py
Momentum	RSI, MACD, MA distances, rate-of-change	Technical analysis
Volume	Relative volume, OBV, VWAP deviation	Microstructure
Patterns	near_support, near_resistance, triangle, channel, MTB	scanner.py
Regime	HMM state label, transition probability, time-in-regime	Regime model

4. System Architecture

The system follows a layered pipeline architecture where each module transforms data and passes enriched outputs downstream. The diagram below shows all components, their dependencies, and current implementation status.



4.1 Pipeline Flow



The pipeline proceeds in 6 phases. Phases 0–1 (Data + Patterns) are complete. Phase 2 (Feature Engineering + Regime Detection) is the immediate next step. The remaining phases build on each other sequentially.

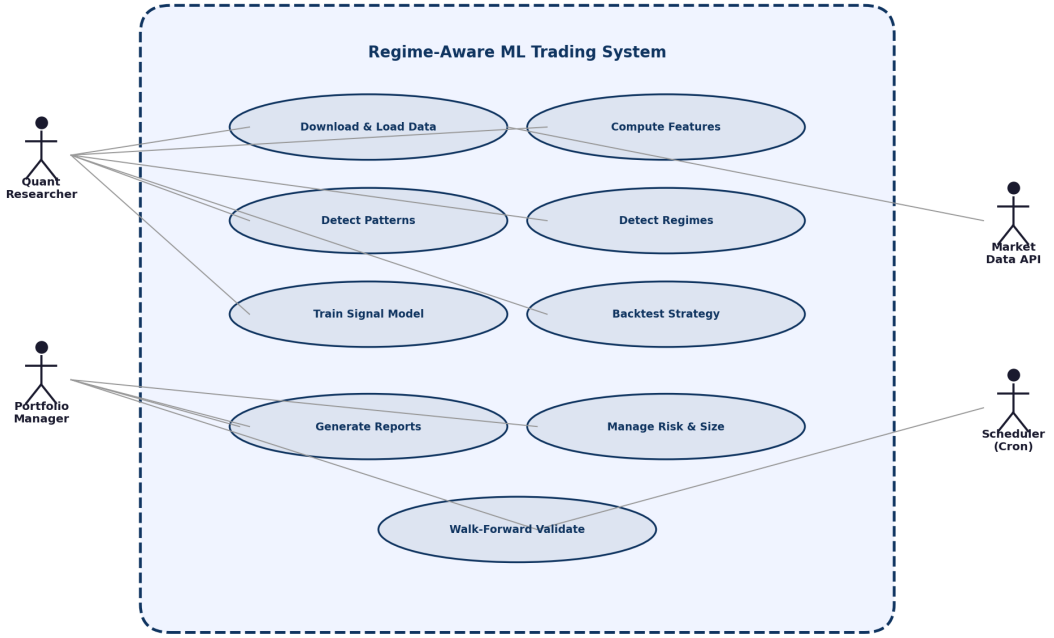
4.2 Module Responsibilities

Module	Directory	Key Files	Status

Data Layer	src/data/	download_data.py, load_data.py, utils.py	Complete
Pattern Detection	src/patterns/	scanner.py, 4 detectors, evaluate_rates.py	Complete
Feature Engineering	src/features/	(placeholder)	Next
Regime Detection	src/regimes/	(placeholder)	Next
Signal Model	src/models/	(placeholder)	Planned
Labeling	src/labeling/	(placeholder)	Planned
Backtesting	src/backtest/	(placeholder)	Planned
Reporting	reports/	3 PDF generators	Ongoing

5. Use Cases

Use Case Diagram — Regime-Aware ML Trading System



ID	Use Case	Actor	Description
UC-01	Download & Load Data	Quant Researcher	Fetch SPY OHLCV from yfinance, clean, validate columns, store CSV
UC-02	Compute Features	Quant Researcher	Generate ATR, momentum, volatility, fractional diff. features
UC-03	Detect Patterns	Quant Researcher	Scan S/R, triangles, channels, multi T/B detectors on OHLCV data
UC-04	Detect Regimes	Quant Researcher	Train HMM on returns+vol, decode regime labels via Viterbi
UC-05	Train Signal Model	Quant Researcher	Train XGBoost on features+regime+patterns with triple-barrier labels
UC-06	Backtest Strategy	Portfolio Manager	Walk-forward validate strategy with realistic slippage model
UC-07	Generate Reports	Portfolio Manager	PDF reports with detection rates, charts, spot-checks, regime analysis
UC-08	Manage Risk & Size	Portfolio Manager	Regime-conditional Kelly sizing, drawdown brakes, VaR limits
UC-09	Walk-Forward Validate	Scheduler	Automated rolling retrain + test cycle for production monitoring

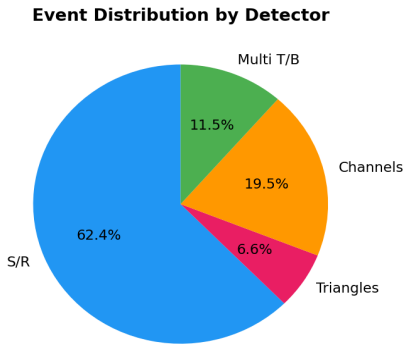
6. Current State & Results

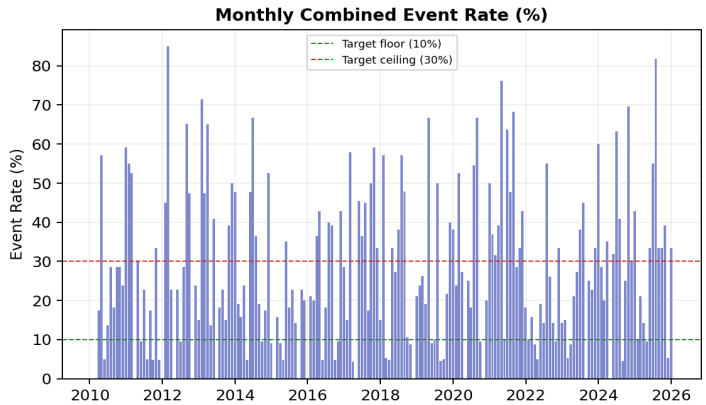
6.1 Completed Work

- **Data pipeline:** yfinance download + CSV loader for SPY (2010–2025, ~4,000 bars).
- **Shared ATR utility:** Wilder's True Range with 14-period SMA, used across all detectors.
- **4 pattern detectors** with ATR-based adaptive thresholds, linear regression trendlines, breakout-only firing, and structural validation (touch counts, compression checks).
- **2 iterations of parameter tuning:** combined rate reduced from 45.7% to 27.3%.
- **3 PDF report generators** documenting methodology, tuning, and visual validation.

6.2 Detection Rates (Iteration 2)

Detector	Count	Rate	Iter 1 Rate	Change
Support / Resistance	757	18.8%	28.2%	-9.4pp
Triangles	80	2.0%	2.0%	-0.0pp
Channels	237	5.9%	24.1%	-18.2pp
Multi Top / Bottom	139	3.5%	3.5%	-0.0pp
COMBINED	1098	27.3%	45.7%	-18.4pp





7. Roadmap & Implementation Plan

Phase	Name	Key Deliverables	Dependencies
0	Data Pipeline	yfinance download, CSV loader, ATR utility	None
1	Pattern Detection	4 detectors, scanner, evaluate_rates.py	Phase 0
2a	Feature Engineering	Frac. diff., momentum, vol, volume features CUSUM filter	Phase 0
2b	Regime Detection	GaussianHMM (2-3 states) Viterbi decoding, BIC model selection	Phase 0
3	Trade Labeling	Triple-barrier labels Meta-labeling framework	Phases 1, 2a
4	Signal Model	XGBoost classifier RF baseline, regime-ablation	Phases 2b, 3
5	Risk & Backtest	Walk-forward engine, Kelly sizing Drawdown brakes, CPCV	Phase 4
6	Final Evaluation	Per-regime Sharpe/MDD decomposition Deflated Sharpe, bootstrap CI	Phase 5

7.1 Phase 2a — Feature Engineering (Detailed)

- **Step 1:** Implement fractional differentiation using the `fracdiff` package. Find minimum d that passes ADF test at 95% confidence.
- **Step 2:** Compute momentum features: RSI(14), MACD(12,26,9), rate-of-change at 5/21/63 horizons.
- **Step 3:** Compute volatility features: realised vol (21d), Garman-Klass vol, ATR (already done).
- **Step 4:** Compute volume features: relative volume (vol / 20d avg), OBV, VWAP deviation.
- **Step 5:** Implement CUSUM event filter as an alternative sampling method.
- **Step 6:** Build feature matrix combining all features + pattern signals for model training.

7.2 Phase 2b — Regime Detection (Detailed)

- **Step 1:** Train GaussianHMM with 2, 3, and 4 states on daily log returns + 21d realised vol.
- **Step 2:** Select optimal state count using BIC / AIC.
- **Step 3:** Decode regime labels via Viterbi algorithm. Verify regime persistence (avg duration > 20 bars).
- **Step 4:** Validate: compare regime-conditional return distributions (expect different means and variances).
- **Step 5:** Export regime labels as a feature column for the signal model.

7.3 Phase 4 — Signal Model (Detailed)

- **Step 1:** Generate triple-barrier labels (TP=2xATR, SL=1xATR, expiration=10 bars) for each event bar.
- **Step 2:** Build training set: features + regime + patterns as X, triple-barrier label as y.
- **Step 3:** Train XGBoost classifier with walk-forward splits (3yr train / 6mo test).
- **Step 4:** Train Random Forest baseline for ablation comparison.
- **Step 5:** Ablation study: train same XGBoost **without** regime features to test H1.
- **Step 6:** Evaluate: OOS Sharpe, hit rate, precision/recall, feature importance (SHAP).

8. Recommendations & Suggestions

8.1 Immediate Actions (Next Sprint)

- **Implement feature engineering module** (src/features/). Start with fractional differentiation and momentum features — these have the strongest empirical support.
- **Build HMM regime detector** (src/regimes/). Use hmmlearn.GaussianHMM with 2–3 states. The dependencies (hmmlearn, scikit-learn) are already in requirements.txt.
- **Vectorise triangle & channel loops.** The current O(n×w) bar-by-bar loops work for ~4,000 bars but will not scale. Use numpy stride tricks or rolling regression.

8.2 Data Suggestions

- **Multi-asset expansion:** Add QQQ, IWM, TLT, GLD to test regime detection across asset classes. Cross-asset regime signals are often more robust.
- **VIX as an input feature:** Download ^VIX and include it as a feature — it is forward-looking (implied vol) vs. ATR which is backward-looking (realised vol).
- **Volume bars:** Consider dollar-bar sampling (de Prado) instead of time bars for more information-uniform sampling.

8.3 Model Suggestions

- **Start simple:** Logistic Regression → Random Forest → XGBoost. Each model serves as a baseline for the next. Avoid jumping to deep learning without beating tree baselines.
- **Meta-labeling:** Train a secondary model that predicts bet size (confidence) given the primary model's direction signal. This naturally handles regime-dependent signal quality.
- **SHAP analysis:** Use SHAP values to explain which features drive predictions in each regime. This provides interpretability and helps debug the model.
- **Ensemble with HMM:** Consider the Gupta et al. (2025) hybrid approach: HMM + ensemble ML voting classifier for regime detection.

8.4 Risk & Backtest Suggestions

- **Use VectorBT** for backtesting — it is the fastest Python framework and supports vectorised operations natively. Ideal for parameter sweeps.
- **Implement the Deflated Sharpe Ratio** (Bailey et al., 2017) to control for multiple-testing bias when comparing strategies.
- **Regime-conditional position sizing:** Reduce to half-Kelly in bear regimes. Your ATR utility already provides the volatility measure for stop placement.
- **Walk-forward with embargo:** Use 3-year train / 6-month test windows with a 5-day embargo between train and test to prevent label leakage.

8.5 Thesis-Level Contributions

To elevate this project to thesis quality, focus on these original contributions:

- **Regime-ablation study:** Rigorously compare ‘same model with regime’ vs ‘same model without regime’ on identical walk-forward splits. This directly tests H1 and is the core contribution.
- **Per-regime performance decomposition:** Report Sharpe, MDD, hit rate separately for bull, bear, and sideways regimes. Show that the system adapts its behaviour.
- **Bootstrap confidence intervals:** Use block bootstrap to compute confidence intervals for all metrics, establishing statistical significance.
- **Transaction cost sensitivity:** Show how results degrade across different slippage assumptions (5bps, 10bps, 20bps).

9. References

[1] Hamilton, J.D. (1989). “A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle.” *Econometrica*, 57(2), 357–384.

[2] Nystrup, P., Kolm, P.N. & Lindstrom, E. (2020). “Greedy Online Classification of Persistent Market States Using Realized Intraday Volatility Features.” *Journal of Financial Data Science*, 2(3).

[3] Gupta, R., Kapoor, S., Gupta, H. & Natesan, S. (2025). “A Forest of Opinions: A Multi-Model Ensemble-HMM Voting Framework for Market Regime Shift Detection.” *Data Science in Finance and Economics*, 5(4): 466–501.

[4] Lo, A.W., Mamaysky, H. & Wang, J. (2000). “Foundations of Technical Analysis: Computational Algorithms, Statistical Inference, and Empirical Implementation.” *Journal of Finance*, 55(4), 1705–1765.

[5] Brock, W., Lakonishok, J. & LeBaron, B. (1992). “Simple Technical Trading Rules and the Stochastic Properties of Stock Returns.” *Journal of Finance*, 47(5), 1731–1764.

[6] de Prado, M.L. (2018). *Advances in Financial Machine Learning*. Wiley.

[7] Bailey, D.H., Borwein, J.M., de Prado, M.L. & Zhu, Q.J. (2017). “The Probability of Backtest Overfitting.” *Journal of Computational Finance*.

[8] Ang, A. & Bekaert, G. (2002). “International Asset Allocation with Regime Shifts.” *Review of Financial Studies*, 15(4), 1137–1187.

[9] Thorp, E.O. (2006). “The Kelly Criterion in Blackjack, Sports Betting, and the Stock Market.” In *Handbook of Asset and Liability Management*.

[10] Aydinhan, A., Kolm, P.N., Mulvey, J.M. & Shu, Y.O. (2024). “Identifying Patterns in Financial Markets: Extending the Statistical Jump Model.” *Annals of Operations Research*.

[11] Ibanez, F.A. (2024). “Incorporating Market Regimes into Large-Scale Stock Portfolios.” MPRA Paper No. 121552.

[12] Lo, A.W. (2004). “The Adaptive Markets Hypothesis.” *Journal of Portfolio Management*, 30(5), 15–29.

[13] RegimeFolio (2025). “Regime-Based Sector-Specific Portfolio Optimization.” Arxiv 2510.14986 / IEEE Access.

[14] Sullivan, R., Timmermann, A. & White, H. (1999). “Data-Snooping, Technical Trading Rule Performance, and the Bootstrap.” *Journal of Finance*, 54(5).